INTERNATIONAL UNIVERSITY OF APPLIED SCIENCES

DLMBIRND01 RELATIONAL AND NOSQL DATABASES

WRITTEN ASSIGNMENT

Memory Management in Databases

Author: Tardzenyuy Emmanuel Yufenyuy

Enrollment Number: IU14091118

Study Program: Business Intelligence

Table of Contents

Abstract 0

Introduction 1

Key Terminologies 3

Shared Storage Management Technique and Concept 3

- **Scalability** 3
- **The CAP Theorem** 4

PostgreSQL 5

- **Concepts and Features for Evaluating PostgreSQL** 5
  - ***Data Model*** 5
  - ***ACID*** 6
  - ***Scalability*** 7
  - ***Data Partitioning and Replication*** 7
  - ***Indexing*** 8
  - ***Application/Use Case*** 9

Apache Cassandra 9

- **Concepts and Features for Evaluating Apache Cassandra** 9
  - ***Data Model*** 10
  - ***Scalability*** 10
  - ***Data Partitioning and Replication*** 11
  - ***Indexing*** 11
  - ***Application / Use Case*** 12

Analysis 13

Recommendations and Conclusion 15

Table of Figures

Fig 1: Compare and Contrast Apache Cassandra and PostgreSQL..... 14
Fig 2: Structural Advantage and Disadvantage of PostgreSQL..... 14
Fig 3: Structural Advantage and Disadvantage of Apache Cassandra. 15

Abstract

Nowadays a platitude of data is being rapidly generated from numerous sources including social media, telecommunication systems, machine sensors etc, and across different organizations including healthcare, logistics, transport, finance, media etc. In this literature review, memory management will be investigated in two database management systems; one relational database system(PostgreSQL) and one nonrelational database system(Apache Cassandra). The review will explore their respective domains of application, classical use cases, built-in features and underlying paradigms or concepts such as ACID, BASE, CAP, Normalization etc. Furthermore, the review will dive into data storage management techniques including Scaling, Replication and Partitioning, clearly demonstrating their implementation and importance in the chosen database systems. A comparative analysis of the structure of these database systems based on the concepts, storage techniques and features provide insight into the impact of implemented storage approaches on performance. The findings of this literature review should provide users with scientifically backed reasons for the deployment of these database systems with regard to business requirements. The rest of this paper is structured as follows: A detailed introduction on the research topic will be followed by a description of some key terminologies that might be used across the entire review task. Then an elaboration of key concepts or storage techniques applicable to chosen database management systems. These techniques will further be explained in how they are implemented by each of the aforementioned systems. Furthermore, PostgreSQL will be discussed alongside its use cases and the implementation of some of its features and storage management techniques for performance improvement. The latter will be done for Apache Cassandra as well. This will then be proceeded by a table comparing and contrasting these database management systems, as well as a tabular presentation of the structural advantages and disadvantages of each of these systems. This literature review will be closed by providing recommendations and concluding statements.

Introduction

Database management Systems (DBMSs) are used to efficiently manage increasing volumes of data and perform data analytics in different application areas (Manos Frouzakis et al., 2025). Handling these growing large amounts of data can be quite challenging especially when the DBMS performs operations that include data retrieval, aggregation, ordering and join (Manos Frouzakis et al., 2025). This challenge is as a result of significant memory consumption and storage which has an adverse effect on query performance (Simon Lindgren, 2024). These operations form the basis for evaluating the performance of commercial database management systems in TPC-H Benchmarking (Manos Frouzakis et al., 2025). With the introduction of main memory databases, it is possible to store entire databases on large memory sizes to improve execution time and energy consumption even though this does not completely alleviate data movement bottlenecks (Manos Frouzakis et al., 2025). There exist several processing architectures for memory management including processor-centric architecture and data-centric architecture. Processor-centric architectures like CPUs, GPUs and FPGAs can improve memory usage in DBMSs by physically arranging hardware components close to each other or by deploying high-bandwidth memory(HBM) devices (Manos Frouzakis et al., 2025). In a more recent approach like data-centric architecture such as processing-in-memory (PIM), computation resources are deployed inside the main memory where the data resides (Manos Frouzakis et al., 2025). Nowadays, it is necessary to differentiate between relational databases and nonrelational databases. In (E. F. CODD, 1990) one of the descriptions of the word **relation** according to the Oxford English Dictionary states that it is “That feature or attribute of things which is involved in considering them in comparison or contrast with each other”. This description is prone to precision concerns in communication as it stems from a natural language (E. F. CODD, 1990). For the sake of generalization except otherwise needed for specific scenarios, the term **relation i.e R,** is understood in mathematics as a set of n-tuples derived from the cartesian product of m-sets (E. F. CODD, 1990). Tuples in a mathematical relation are *unordered and of the same type*. These two properties of tuples in a mathematical relation form the building blocks of the relational model (E. F. CODD, 1990). Thus a relational table of a relational model is made up of unordered distinct tuples or rows where every column is named differently from the other and is accessible by its unique name rather than by an index or number (E. F. CODD, 1990). Additionally, the columns in a table or relational table are unordered and every row or tuple is identifiable via an identification key (Andreas Meier & Michael Kaufmann, 2019). An identification key, in other words primary key, of a relational table is a column or minimal concatenation of columns whose entries uniquely identifies rows within this table (Andreas Meier & Michael Kaufmann, 2019). It is on the concept of the relational model that relational databases are developed. A relational database is a collection of relational tables where data is stored atomically (E. F. CODD, 1990). Several

languages have been developed to access and manipulate data stored in relational databases. Examples of these languages are Query Language (QUELL), Structured Query Language (SQL) etc (E. F. CODD, 1990). SQL was developed by a group of researchers at IBM in 1972 and has been accepted as the ANSI (American National Standard Institute) and ISO(International Organization for Standardization) Standard given that it is also highly supported by many database management system developers (E. F. CODD, 1990);(Andreas Meier & Michael Kaufmann, 2019). Examples of relational database management systems are Oracle, MySQL, Microsoft SQL Server, PostgreSQL, Db2, MariaDB etc, (Gianina Mihai, 2020). As a result of large amounts of data from heterogeneous sources e.g social media posts, stock markets, E-commerce, Satellite images, Health care etc, the term big data has emerged without a generally accepted definition (Andreas Meier & Michael Kaufmann, 2019). However big data is widely accepted as a term that constitutes **volume**(huge amounts of data), **variety**(data in different formats i.e structured e.g employee table, semi-structured e.g JSON files, and unstructured e.g audio records), **velocity**(data delivered at high speeds or in real-time), **veracity**(faulty data) and **value**(added advantage derived from data) (Andreas Meier & Michael Kaufmann, 2019). It is in the advent of the challenges, i.e volume, variety, velocity and variety, brought forth by big data, that a family of technologies or databases that made its first appearance in 1998, is strongly resurfacing today as a solution to handle these challenges given that relational databases are not equipped to deal with these challenges (Andreas Meier & Michael Kaufmann, 2019). This family of technologies fall under the NoSQL(Not only SQL) umbrella (Andreas Meier & Michael Kaufmann, 2019). The key strengths of NoSQL database management systems lie in the fact that they are open source, nonrelational, horizontally scalable and distributive in nature (Gianina Mihai, 2020). Additionally they are split into four families or categories, namely: **Key-Value** stores(e.g. Redis, Riak KV), **Document** stores (e.g. MongoDB, CouchDB etc), **Wide Column** stores (e.g. Apache Cassandra, Apache HBase etc) and **Graph** stores (e.g. Neo4j, OrientDB etc) (Gianina Mihai, 2020). There also exists NoSQL databases that implement all the properties of the aforementioned families e.g. Azure Cosmos DB etc (Andreas Meier & Michael Kaufmann, 2019). In this research task, PostgreSQL as a relational database management system will be compared and contrasted with Apache Cassandra from the wide column family of NoSQL database management systems. To compare and contrast these two database management systems will be evaluated or examined with regards to the following criteria: Underlying Paradigm(data model), Transactions, Query Language, CAP-theorem, BASE, Data Structure, Scalability, Cost, Application/usage (Felix Gessert et al., 2016; Gianina Mihai, 2020). Through this a clear overview on the similarities and differences of these databases will be provided.

Key Terminologies

- Database: This is electronically stored data that is collected in a logically organised form. This data can typically be directly retrieved and viewed (Andreas Meier & Michael Kaufmann, 2019)
- Database Management System (DBMS): They store and manage data. In the relational model, the storage component is responsible for storing data, metadata, and the relationships defined between the individual relational tables. The management component in the relational model is in charge of the database design as well as the language, in this case SQL, used for the data definition/declaration, data manipulation and querying (Andreas Meier & Michael Kaufmann, 2019). The management component is equally responsible for protecting and backing up the data (Andreas Meier & Michael Kaufmann, 2019). On the other hand, the storage component in nonrelational databases mostly leverages an architecture for massively distributing the stored data. As earlier mentioned the data can be stored as key-pair, in wide column families, as documents, and as graphs (Andreas Meier & Michael Kaufmann, 2019)
- Synchronous Replication: A method of prioritizing data consistency, at the expense of performance, by committing transactions only when data has been copied to all the nodes of the same cluster (Vougiouklis Evangelos, 2023).
- Asynchronous Replication: A method of prioritizing performance, at the expense of data consistency, by allowing a node to commit its transactions before data is copied to the next nodes of the same cluster (Vougiouklis Evangelos, 2023).

Shared Storage Management Technique and Concept

Scalability

The aspect of adding more computational resources to a database management system to support its performance in response to increasing workloads, without the need to modify the structure of the database system (Didar Yedilkhan et al., 2023). Scalability can be leveraged horizontally or vertically. Horizontal scalability is a practice that involves an increase in the number of nodes to manage increasing workloads and data volumes (Didar Yedilkhan et al., 2023). On the other hand, vertical scalability is leveraged by adding more computational resources to one or more servers of the database system (Didar Yedilkhan et al., 2023). Typically, vertical scalability has a threshold to the computing resources that can be added to a server and the associated financial cost of adding resources is disproportionate to the performance benefits (Didar Yedilkhan et al., 2023). This gives horizontal scalability a significant edge over vertical scalability. However, horizontal scalability also poses challenges in relational database management systems that were initially designed to function on a single server but now are expected to be suitable for both clustering and cloud environments (Didar

Yedilkhan et al., 2023). According to (Didar Yedilkhan et al., 2023). Continuous research efforts are being made to improve efficiency, scalability, and versatility in NoSQL technologies to make them more suitable for managing big data applications.

The CAP Theorem

As elaborated in the description of a DBMS, storage is an essential component of all DBMSs. However, another relevant property of a database system, that must be discussed, is the level of consistency it provides (Felix Gessert et al., 2016). While some database systems are designed to be *strictly consistent and serializable* summarized under the ACID(Atomicity, Consistency, Isolation, Durability) constraint, others are built to be *available* i.e BASE (Basically Available, Soft-state, Eventually Consistent) (Felix Gessert et al., 2016). The trade-off between consistency and availability has been observed in several NoSQL databases and distributed systems, and the CAP(Consistency, Availability, Partitioning) theorem has proven its effectiveness in classifying NoSQL databases based on this trade-off in the event of network partitioning (Andreas Meier & Michael Kaufmann, 2019). In a broader sense, a system can only guarantee two of the three properties **Consistency, Availability and Partition-tolerance** during read/write operations (Felix Gessert et al., 2016).

- Consistency (C): Every node in a cluster of nodes has the same data at all times. This means that the same query from different clients must yield the same results. (Felix Gessert et al., 2016).
- Availability (A): Every active node in a cluster of nodes must respond to every read/write request from clients, and the response must not be an error (Felix Gessert et al., 2016).
- Partition-tolerance (P): In the event of message loss between nodes or failure of some nodes, then the system must ensure either consistency or availability.

In the event of a network partition, a system is eventually consistent if it prioritizes availability (AP systems) and strictly consistent if it prioritizes consistency(CP Systems) (Felix Gessert et al., 2016). Single-node systems ensure consistency and availability(CA) but fail should the system be partitioned (Felix Gessert et al., 2016). It has been shown that the CAP theorem does not address the latency and consistency trade-off during normal operations of NoSQL systems (Felix Gessert et al., 2016). However, the attempt to address this trade-off using the PACELC theorem is not successful as the PACELC theorem does not effectively classify NoSQL as the CAP does (Felix Gessert et al., 2016). The PC/EL class of the PACELC theorem cannot be assigned to any database system (Felix Gessert et al., 2016). The PACELC-theorem states that there is an availability-consistency trade-off in case of partition, or else a latency-consistency trade-off in normal operation.

PostgreSQL

This is an object-oriented relational database management system that was created in 1986 at the University of California in Berkeley as part of their POSTGRES project (*PostgreSQL Global Development Group*, 2025). Today this powerful database management system is actively being developed and it is free and open source under the PostgreSQL License (*PostgreSQL Global Development Group*, 2025). Besides being compliant with the ACID principle, it runs on Windows, mcOS, Linux, BSD and Solaris (*PostgreSQL Global Development Group*, 2025). Programming languages including Java, Python, Ruby, Perl, C++ etc. can interact with PostgreSQL (Simon Lindgren, 2024). The query language in PostgreSQL is SQL with wide support for complex queries, subqueries, joins, window functions and common table expressions (CTEs) (Yijie Weng & Jianhao Wu, 2025). This DBMS supports custom datatypes besides primitive data types(e.g Numeric, string, Boolean, Date/Time etc.), Document (e.g JSON/BSONB), Key-Value(Hstore) etc (*PostgreSQL Global Development Group*, 2025). To understand how PostgreSQL manages its memory for performance enhancement, some concepts and features with regard to PostgreSQL will now be examined and elaborated.

Concepts and Features for Evaluating PostgreSQL

In this segment, a literature review explaining the use of concepts and features for memory management and performance improvement in PostgreSQL is done. This constitutes academic research, experiments and applications elaborating the implementation of these concepts and features in PostgreSQL to provide a comprehensive understanding of how memory can be managed in PostgreSQL and to improve performance.

Data Model

Since PostgreSQL is an object-oriented relational database management system, it thus supports the relation model. This means that PostgreSQL stores data as relations or relational tables. As elaborated in (Lacey-Anne Sanderson et al., 2021), the data was stored in PostgreSQL in the form of a schema that was derived from an entity-relationship-diagram i.e a model which is used to represent important tables of the situation in question. The relationships between the tables involved were implemented by the use of referential integrity (Lacey-Anne Sanderson et al., 2021). This flexible and efficient approach to storing data in PostgreSQL ensures data integrity while the data stays normalized (Lacey-Anne Sanderson et al., 2021). Another advantage of storing data in relationships is that it reduces storage space immensely and eliminates duplicated records (Lacey-Anne Sanderson et al., 2021). Normalization is an important concept in relational databases as it provides a formal mechanism by applying functional dependency to check whether fields/columns are in the right table, otherwise, the tables will be restructured (Clare Churcher, 2012). Examples of issues

that can be resolved with normalization are redundancy, insert and delete anomalies (Clare Churcher, 2012).

ACID

ACID is an acronym for Atomicity, Consistency, Isolation and Durability. Jointly, these words form a data integrity constraint that ensures *that multiple users* may execute *transactions* on a database management system *without any transaction conflicting* the other (Andreas Meier & Michael Kaufmann, 2019). A transaction is thus a set of operations that is bound by atomicity, consistency, isolation and durability (Andreas Meier & Michael Kaufmann, 2019). According to (Yijie Weng & Jianhao Wu, 2025) PostgreSQL is ACID-compliant, which means that transactions that fulfill atomicity, consistency, isolation and durability will maintain their validity irrespective of the number of transactions simultaneously made or whether there is a power outage (Lacey-Anne Sanderson et al., 2021). According to (Lacey-Anne Sanderson et al., 2021), there are constraints within PostgreSQL that can be used to ensure that data types, relationships and boundaries are implemented. For instance, PostgreSQL leverages triggers and stored procedures to enhance the implementation of complex business logic thereby ensuring data integrity and security (Vougiouklis Evangelos, 2023). Transactions occurring concurrently can be effectively managed via its Multi-Version Concurrency Control (MVCC) feature which ensures data consistency (Vougiouklis Evangelos, 2023).

- Atomicity: A transaction must be fully and successfully executed otherwise it must be completely *reverted/ rollbacked* to the previous state without leaving any traces (Andreas Meier & Michael Kaufmann, 2019).
- Consistency: This ensures that the database system must be consistent before and after the complete execution of a transaction (Andreas Meier & Michael Kaufmann, 2019).
- Isolation: The database system must *serialize* parallel transactions and ensure that these transactions are executed independently from each other (Andreas Meier & Michael Kaufmann, 2019).
- Durability: The state of the database system must only change at the end of a successfully executed transaction (Andreas Meier & Michael Kaufmann, 2019). Hence, the state of the database system must not be affected by software errors, system failures, or errors on external storage media (Andreas Meier & Michael Kaufmann, 2019).

Scalability

PostgreSQL is foundationally designed for vertical scalability and offers outstanding performance in handling complex queries and transactions even though it can be overwhelmed by large amounts of data that require write operations (Yijie Weng & Jianhao Wu, 2025). To achieve horizontal scalability in PostgreSQL, the Citus extension can be leveraged as it provides a distributed architecture that allows data table partitions to be distributed across several nodes or shards which is suitable for parallel query execution and high performance (Vougiouklis Evangelos, 2023). In an experiment conducted by (Didar Yedilkhan et al., 2023) to evaluate the impact of scaling data as a measure of time and capacity for Insert, Search and Delete operations, the following observations can be made about PostgreSQL.

- Insert: Insertion time grew exponentially at huge amounts of data when inserting in an empty database as opposed to a database that contains data. However, insertion time in a database containing data appeared to be constant for all sizes of data. The capacity of PostgreSQL to manage increasing amounts of data during insertion was fairly constant in both instances.
- Search: With huge volumes of data, PostgreSQL exhibited a significant increase in search time and its capacity to manage data significantly drops with increasing volumes of data.
- Delete: As data grew, PostgreSQL exhibited a significant increase in delete time but its capacity to manage data was similar to its capacity to manage data in search operations.

Data Partitioning and Replication

Data replication in PostgreSQL involves copying data from a primary PostgreSQL instance to one or more instances to guarantee access to data and provide data consistency (Vougiouklis Evangelos, 2023). Replication methods supported in PostgreSQL include Synchronous, Asynchronous, Streaming and Logical replication (Vougiouklis Evangelos, 2023).

- Streaming replication: A feature in PostgreSQL that enables the continuous copying of data from the primary node to one or more nodes, within the same cluster, in near real-time (Vougiouklis Evangelos, 2023).
- Logical replication: In PostgreSQL, this method reflects changes in data at the query level without persisting these changes on the database (Vougiouklis Evangelos, 2023).

(Vougiouklis Evangelos, 2023) demonstrated how PostgreSQL's internal features can be used to implement logical, asynchronous and synchronous replication. The focus was on the behavior of PostgreSQL in asynchronous and synchronous replication in the event that non-

primary nodes or replicas fail. When the replicas fail in synchronous replication, the primary node stops and restrains every transaction until the replicas become available (Vougiouklis Evangelos, 2023). On the other hand, when the replicas fail in asynchronous replication, the primary node continues with its operations and tracks all changes in a Write-Ahead-Log (WAL) segment. As a result, there is a risk of losing data in asynchronous replication (Vougiouklis Evangelos, 2023). To avoid data loss, a series of measures can be adopted which include, well-defined backup and recovery procedures, the provision of sufficient WAL to the primary node, and the implementation of monitoring systems (Vougiouklis Evangelos, 2023). As already mentioned in PostgreSQL's scalability, data partitioning can be achieved by deploying the Citus extension (Vougiouklis Evangelos, 2023).

Indexing

These are data structures that are bonded to database tables and used to quickly find rows based on specific field values to improve query performance (Simon Lindgren, 2024). An index of an attribute efficiently organises the records in this attribute and assigns unique addresses to them for quick access (Andreas Meier & Michael Kaufmann, 2019). According to (*PostgreSQL Global Development Group*, 2025) the database management system looks up every row of an entire table in search of records that match the specified search criteria. This is an inefficient approach (*PostgreSQL Global Development Group*, 2025). With the creation of an index on one or more fields or columns, the database system might take a shorter path in the search process to find matching records of the search condition (*PostgreSQL Global Development Group*, 2025). As a result, indexes can improve the database system's performance even though they can overwhelm the database system when used without caution (*PostgreSQL Global Development Group*, 2025). Indexes in PostgreSQL can also have a positive impact on conditional update and delete operations, and can significantly improve join operations that are performed on fields with indexes (*PostgreSQL Global Development Group*, 2025). PostgreSQL offers several types of indexes that can be created on one or multiple columns. These indexes B-Tree, Hash, Gist, SP-GiST, GIN and BRIN (*PostgreSQL Global Development Group*, 2025). Unless otherwise specified, PostgreSQL uses the B-tree index by default when creating indexes. This is because this index type is suitable for most common use cases (*PostgreSQL Global Development Group*, 2025). A B-tree is self-balancing and all nodes, except the root, can have more than two subtrees (Andreas Meier & Michael Kaufmann, 2019). This self-balancing property is useful for keeping the height of the tree low while creating more branches and therefore positively impacting the access time for individual keys (Andreas Meier & Michael Kaufmann, 2019). In an experiment conducted by (Simon Lindgren, 2024) on different sizes of data based on CRUD (Create, Read, Update, Delete) operations for insert, select, update and remove using B-tree index, results showed that PostgreSQL performed faster with this index than without. Additionally, the

experiment also showed that more memory is used up when this index is used on large(e.g >= 10^7) datasets. This emphasises the need to consider situations before leveraging indexes.

Application/Use Case

PostgreSQL can be used to manage financial transactions given that it is ACID-compliant. PostgreSQL is being used by well-doing companies like Twitch, Spotify, Reddit and IMBD (Simon Lindgren, 2024). (*AlloyDB*, 2025) is created and managed by Google Cloud and is an object-oriented relational database that is not only completely forked from PostgreSQL but is also fully compatible with PostgreSQL. According to the discussion in (Merv Adrian, 2024), Apex Fintech Solutions leverages AlloyDB for PostgreSQL to benefit from its horizontal and vertical scalability besides its high availability to manage their financial transactions and calculations. PostgreSQL can equally be used for the storage of large-scale biological data where data integrity and query performance are vital factors. This is because of the possibility to store data in a schema and the use of materialized views as well as indexes to enhance storage and improve query performance respectively (Lacey-Anne Sanderson et al., 2021). PostgreSQL is highly reliable and ensures strong data integrity, and accuracy which is relevant in data analytics (Christian Coronel-Guerrero et al., 2024).

Apache Cassandra

Cassandra was created by Facebook and belongs to the wide-column store family of NoSQL technologies (Elif Dede et al., 2013). In addition to being non-relational and open source, it is distributive and is built on a peer-to-peer model for robustness including its ability to scale horizontally (Elif Dede et al., 2013). Cassandra is suitable for storing structured and unstructured data (Christian Coronel-Guerrero et al., 2024). The query language is Cassandra Query Language(CQL) which is similar to SQL but in addition to data query, it only supports data definition and data manipulation (Apache Software Foundation, 2025). It supports a variety of data types, for instance, native types, collections, tuples, user-defined and custom types (Apache Software Foundation, 2025). With CQL several functionalities including dynamic data masking, indexing and creation of triggers can be achieved (Apache Software Foundation, 2025).

Concepts and Features for Evaluating Apache Cassandra

A similar approach to PostgreSQL is applied to Apache Cassandra to examine the use of concepts and features in Cassandra to enhance memory management and improve performance.

Data Model

Cassandra stores data in columns where a column is the smallest storable component of data which must have a name, value and typically with an associated time stamp (Elif Dede et al., 2013). A collection of columns, where each value in the individual columns is identifiable by a corresponding row key, can be stored in a column family (Elif Dede et al., 2013). Additionally, column families share a similarity with relational data tables and the data stored in column families are arranged in order of the row key (Elif Dede et al., 2013). Data in the same column family are stored together on disk, thus permitting efficient data compression during storage and fast data retrieval (Felix Gessert et al., 2016). Furthermore, column families are stored in *keyspaces*. A keyspace can be considered as a schema in relational database management systems (Elif Dede et al., 2013). An additional advantage of storing data in columns is that *null values* don't take up any space on the disk and data queries that involve different column families are efficient because only a few servers might be involved in the process (Felix Gessert et al., 2016). In wide-column stores, it is important to differentiate between row keys and column keys. Row keys are first level keys while column keys are second level keys (Felix Gessert et al., 2016).

Scalability

Cassandra supports horizontal scalability (Christian Coronel-Guerrero et al., 2024). In the same experiment conducted by (Didar Yedilkhan et al., 2023), Cassandra exhibited the following properties during Insert, Search and Delete operations.

- Insert: Insertion time significantly grew at huge amounts of data when inserting in an empty database as opposed to a database that contains data. However, insertion time in a database containing data appears to be constant for all sizes of data. The capacity of Cassandra to manage increasing amounts of data during insertion was fairly constant in both instances.
- Search: Cassandra had a fairly constant search time irrespective of the amount of data in the database. Its ability to handle large amounts of data in search operations significantly improved. For smaller amounts of data, its capacity is less than that of PostgreSQL.
- Delete: Cassandra has a fairly constant and fast delete time irrespective of the amount of data in the database. Its ability to handle large amounts of data in delete operations strongly improved from small to large amounts of data.

In this experiment, Cassandra performed overall better than PostgreSQL. Another experiment conducted by (Christian Coronel-Guerrero et al., 2024) on Yahoo Cloud Serving Benchmark (YCSB) further confirms the superiority of Cassandra over PostgreSQL in terms of scalability.

Data Partitioning and Replication

Following the CAP theorem, Apache Cassandra is an Availability Partition-tolerance (AP) system (Felix Gessert et al., 2016). This means that, in case of a network partition, the system will continue processing user requests even though there is at least one node that has lost contact with other nodes (Felix Gessert et al., 2016). As a result, AP Systems like Cassandra are eventually consistent (Felix Gessert et al., 2016), i.e. there is a time when the system achieves consistency. Cassandra supports a peer-to-peer framework where every node in a cluster of nodes has equal read and write capabilities, as opposed to the master-worker strategy (Christian Coronel-Guerrero et al., 2024). The peer-to-peer strategy allows Cassandra, through a *replication factor* that can be set by users, to automatically replicate or copy the same data across all the nodes in the cluster (Christian Coronel-Guerrero et al., 2024; Elif Dede et al., 2013). This property ensures high availability or fault tolerance in case of node failures (Christian Coronel-Guerrero et al., 2024), and also allows the nodes to be easily added to a cluster with respect to an increase in data size (Elif Dede et al., 2013). Cassandra also provides the possibility to set the level of latency-to-consistency required for read and/or write operations. The available consistency options that can be set include *zero, one, quorum, all or any* (Elif Dede et al., 2013). A consistency level of *one* implies that at least one node must successfully commit a write operation while it is enough for one node to respond to a read request from a client (Elif Dede et al., 2013). Cassandra supports lazy or asynchronous replication which means that changes are first made on the receiving node before being propagated to other nodes in any order (Felix Gessert et al., 2016). According to (Elif Dede et al., 2013), Cassandra can implement one of two partitioning approaches, which are *RandomPartitioner and ByteOrderPartitione*r (Elif Dede et al., 2013). According to (Elif Dede et al., 2013) *RandomPartitioner*, the default strategy, implements a hash algorithm, thus called **hash-sharding** (Felix Gessert et al., 2016). In hash-sharding, the system partitions every data item across several nodes or machines based on the hash-value determined by a hash-function applied to the primary key (Felix Gessert et al., 2016). The advantages of this approach is that, data is evenly partitioned across all the shards/nodes/machines and only search operations are allowed (Felix Gessert et al., 2016). Through this partition approach, the load can be balanced over all the nodes which is associated to improved performance (Elif Dede et al., 2013).

Indexing

The querying capabilities of Cassandra including other NoSQL databases depend on their level of consistency, distribution and data model (Felix Gessert et al., 2016). (Apache Software Foundation, 2025) supports three types of indexes which are: Primary indexing, Storage-attached indexing (SAI) and Secondary indexing (2i). While the primary and secondary indexes are supported by all versions of Cassandra, SAI is currently only supported

by Version 5.0, which is the latest version of Cassandra (Apache Software Foundation, 2025). Fast and efficient lookup of data matches can be achieved in Cassandra by the use of primary indexes. This is ascertained by the fact that Cassandra deploys hash-sharding, built on the primary key, for data distribution across nodes, as described in the later feature. In concrete terms, all lookup queries based on the primary key can be treated as filter queries that can be implemented in the form of filtered full-table scans. In this case, each node in a Cassandra network performs the designated scan and the results of the scans from the individual nodes are merged and returned (Felix Gessert et al., 2016). According to (Felix Gessert et al., 2016), this is called a *scatter-gather* approach or *MapReduce* as described in (Elif Dede et al., 2013). Secondary indexes are local indexes stored in hidden separate tables from the table containing the indexed values. These index tables are equally stored and managed on each partition of a Cassandra network (Apache Software Foundation, 2025; Felix Gessert et al., 2016). According to (Apache Software Foundation, 2025), secondary indexes should be used alongside a portioning key.

Application / Use Case

Cassandra is used by big tech companies like Netflix, Apple and Instagram due to its capability of storing huge volumes of structured and unstructured data (Christian Coronel-Guerrero et al., 2024). This implies that Cassandra is highly scalable, and thus less costly to maintain (Elif Dede et al., 2013). Additionally, Cassandra is fault tolerant ensuring high availability even in the event of node failures (Christian Coronel-Guerrero et al., 2024). As a result, Cassandra is best suited for big data applications and scenarios where system availability is of utmost importance. A specific use case of Cassandra, with four nodes, is in the development of an Android news application called SmartNews that streams and processes live data from several news outlets for the provision of news in different categories near real-time (Shubham Dhingra et al., 2017). Data was constantly extracted in JSON format with Python from numerous news sources including BBC, ESPN, The Times of India etc, and inserted into a Cassandra node using a Cron Job (Shubham Dhingra et al., 2017). Using a clustering and classification machine learning algorithm, the data in Cassandra was categorized into Business, Technology, Science and Nature, Sports etc (Shubham Dhingra et al., 2017). This application is user-friendly as users can easily search and view, refresh, swipe views etc. SmartNews can also be personalized according to user's preferences and provide the means to perform graphical data analysis (Shubham Dhingra et al., 2017). To demonstrate the high availability or *No Single Point of Failure* of Cassandra, two of the four replicas used were intentionally shut down (Shubham Dhingra et al., 2017). The results showed that the system was not only available but the expected data was accessible via the remaining two nodes that were in production (Shubham Dhingra et al., 2017).

Analysis

In this sector, PostgreSQL and Apache Cassandra will be compared and contrasted based on the concepts and features elaborated above and some internal features of these database management systems that were mentioned. Additionally, (*DB-Engines*, 2025) was also considered to compare and contrast these database systems. (*DB-Engines*, 2025) is a platform that collects and presents vital information on database management systems with emphasis on properties, concepts and systems related to both SQL and the NoSQL movement. To compare and contrast the properties of database systems at a glance, it is possible to place the targeted systems side-by-side (*DB-Engines*, 2025). The decision to consider the properties and concepts presented by (*DB-Engines*, 2025) to compare and contrast PostgreSQL and Apache Cassandra is to support and ascertain the criteria that have been elaborated in this research and to highlight those properties or concepts that have not been covered. In this section, the structural advantages and disadvantages of PostgreSQL and Apache Cassandra will equally be presented.

Compare and Contrast Apache Cassandra and PostgreSQL			
Concept or Feature	Apache Cassandra	PostgreSQL	Type
License	Open source	Open source	Similar
Main data model	Wide Column store	Relational	Different
Supported data model	Vector DBMS	• Document • Spatial • Graph • Vector DBMS	More different than similar
Schema	flexible	fixed	Different
Query Language	CQL	SQL	Closely related
Indexing	Yes	Yes	Yes
Triggers	Yes	Yes	Yes
ACID constraint for Transactions	No	Yes	Different
Consistency	Zero, One, Quorum, Any, All	Strict Consistency guaranteed by ACID	Different
Scalability	horizontal	• Primarily Vertical • Horizontal via extensions	Different
CAP Components	AP	CA	Different
Partitioning Methods	Hash sharding	Via Citus extension	Different

Replication Methods	Replication factor	• Logical • Streaming • Synchronous • Asynchronous	Different
Normalization	No	Yes	Different
MapReduced	Yes	No	Different
Application Areas	Big Data	Where data integrity and strict consistency is required e.g Finance	Different
Use case example	SmartNews App	Apex Fintech Solutions	
Supported Operating Systems	Windows, Linux, OS X, BSD	Windows, Linux, BSD and mcOS, Solaris etc.	Similar
Programming languages	Java, Python, Go, C, C++, Ruby, Perl etc.	Java, Python, Ruby, Perl, C++ etc	Similar

Fig 1: Compare and Contrast Apache Cassandra and PostgreSQL.

Structural Advantage and Disadvantage of PostgreSQL	
Advantages	**Disadvantage**
Support complex queries e.g joins, subqueries, common table expressions (CTAs) etc.	Schema with fixed relationships between tables causes inflexibility.
Support extension e.g Citus for horizontal scalability.	Scalability is costly to maintain.
ACID Constraint ensures data integrity in executing transactions.	Require SQL expertise to implement business logic to ensure data integrity.
Can be hosted on a single node/machine.	System and data become unavailable in case of node/machine failure.

Fig 2: Structural Advantage and Disadvantage of PostgreSQL.

Structural Advantage and Disadvantage of Apache Cassandra	
Advantages	**Disadvantages**
Schemaless	No support for joins.
Highly available and partition tolerant.	Data consistency is not always guaranteed
Handle structured and unstructured data.	More data types to handle.
Stores data in columns which is optimal for memory management as data can be compressed.	

Suitable for handling large volumes of data.	Increased complexity due to a wide variety of data types.
Distributes data evenly on memory.	

Fig 3: Structural Advantage and Disadvantage of Apache Cassandra.

Recommendations and Conclusion

This literature review task aimed at comparing and contrasting PostgreSQL and Apache Cassandra based on their memory management and the associated performance. To achieve this, different concepts and features of these database management systems were evaluated based on academic research and experiments that have been conducted, as well as the use of official documentation provided by the corporations developing and managing these systems. Despite the capabilities of both systems to store and manage structural data, Apache Cassandra stands out in terms of availability in network partitioning and load balancing while PostgreSQL's adherence to the ACID Constraint guarantees data integrity and consistency. As described in the concept of scalability, vertical scalability has a performance limitation associated with high cost. Additionally, PostgreSQL supports vertical scalability and as a result, it is impacted by this limitation. Further research on achieving high performance at low cost in PostgreSQL when adopting vertical scalability is an important aspect that cannot be understated. Although Apache Cassandra, deployed on a single node, performed comparatively better in the experiment conducted by (Didar Yedilkhan et al., 2023), future works should consider exploiting the full potential of Apache Cassandra by deploying more than one node. Besides demonstrating that an increase in the number of nodes in Cassandra significantly improves its performance time in reading and writing voluminous amounts of data, (Elif Dede et al., 2013) equally reading large amounts of data from Cassandra to Hadoop does not have a significant negative effect on Hadoop. Cassandra and Hadoop are tools that are suitable for Analytics (Felix Gessert et al., 2016). Exploring the combination between Cassandra and PostgreSQL in building data applications could lead to developing efficient solutions, that store and manage large volumes of data, by exploiting the strength of these two database management systems.

Considering that each database management system is designed to resolve particular problems, it entails that the choice of one database system over another is based on a set of desirable features (Felix Gessert et al., 2016; Didar Yedilkhan et al. 2023). In this research task, some concepts and features of the chosen database management systems have been explained to enable users to clearly understand how these systems use these concepts and features to efficiently manage their storage capacity and enhance performance. These concepts and properties can provide users with a solid foundation that should ease their

thought process toward choosing the right database system that will fulfill their long and short-term business needs.

References

AlloyDB. (2025). https://dbdb.io/db/alloydb

Andreas Meier & Michael Kaufmann. (2019). *Models, Languages, Consistency Options and Architectures for Big Data Management*. Springer Fachmedien Wiesbaden GmbH. https://doi.org/10.1007/978-3-658-24549-8

Apache Software Foundation. (2025). *Cassandra Documentation*. https://cassandra.apache.org/doc/latest/cassandra/developing/cql/index.html

Christian Coronel-Guerrero, Rodrigo Bastidas-Chal´ an, Daniel Nu˜ nez-Agurto, & Luis Chica-Moncayo. (2024). *Statistical Description of PostgreSQL and Cassandra in CRUD Operations*. https://doi.org/10.1007/978-3-031-70981-4_15

Clare Churcher. (2012). *Beginning Database Design: From Novice to Professional*. Apress.

DB-Engines. (2025). https://db-engines.com/en/

Didar Yedilkhan, Assel Mukasheva, Dariya Bissengaliyeva, & Yerulan Suynullayev. (2023). *Performance Analysis of Scaling NoSQL vs SQL: A Comparative Study of MongoDB, Cassandra, and PostgreSQL*.

E. F. CODD. (1990). The RELATIONAL MODEL for DATABASE MANAGEMENT. In *The Relational Model for Database Management* (Vol. 2, pp. 26–45). ADDISON-WESLEY PUBLISHING COMPANY.

Elif Dede, Bedri Sendir, Pinar Kuzlu, Jessica Hartog, & Madhusudhan Govindaraju. (2013). *An Evaluation of Cassandra for Hadoop*. 2013 IEEE Sixth International Conference on Cloud Computing.

Felix Gessert, Wolfram Wingerath, Steffen Friedrich, & Norbert Ritter. (2016). NoSQL database systems: A survey and decision guidance. *Springer-Verlag*. https://doi.org/10.1007/s00450-016-0334-3

Gianina Mihai. (2020). *Comparison between Relational and NoSQL Databases*. https://doi.org/10.35219/eai15840409134

Lacey-Anne Sanderson, Carolyn T. Caron, Reynold L. Tan, & Kirstin E. Bett. (2021). *A PostgreSQL Tripal solution for large-scale genotypic and phenotypic data*. https://doi.org/10.1093/database/baab051

Manos Frouzakis, Juan Gomez-Luna, Geraldo F. Oliveira, Mohammad Sadrosadati, & Onur Mutlu. (2025). *PIMDAL: Mitigate the Memory Bottleneck in Data Analytics using a Real Processing-in-Memory System*. https://arxiv.org/abs/2504.01948

Merv Adrian. (2024). IT Market Strategy. *PostgreSQL In Financial Services: Apex Fintech and AlloyDB*. https://itmarketstrategy.com/

PostgreSQL Global Development Group. (2025). https://www.postgresql.org/

Shubham Dhingra, Shreeya Sharma, Parmeet Kaur, & Chetna Dabas. (2017). *Fault Tolerant Streaming of Live News using Multinode Cassandra*. Proceedings of 2017 Tenth International Conference on Contemporary Computing (IC3), 10-12 August 2017, Noida, India, Noida, India.

Simon Lindgren. (2024). *PERFORMANCE COMPARISON BETWEEN TWO RELATIONAL DATABASE MANAGEMENT SYSTEMS*. UMEA UNIVERSITY.

Vougiouklis Evangelos. (2023). *Clustering, Replication and High Availability in Postgresql*. University of Macedonia.

Yijie Weng & Jianhao Wu. (2025). *Database management systems for artificial intelligence: Comparative analysis of postgre SQL and MongoDB*. https://doi.org/10.30574/wjarr.2025.25.2.0586